

Apostila 4

Encapsulamento

Uma das grandes vantagens da orientação a objetos é a possibilidade de se restringir o acesso, a certos componentes da classe. Essa propriedade pode ser usada para controlar, por exemplo, que tipo de valores podem ou não ser usados em um certo atributo, evitando erros no sistema proveniente de valores indevidos, aumentando assim sua estabilidade.

Podemos alterar o acesso à um componente através da palavras chave: **public** para acesso totalmente liberado, **protected** para liberar o acesso dentro do mesmo pacote e para subclasses e **private** para dar acesso somente a outros componentes da mesma classe.

```
1 package figurasplanas;
2
3 public class quadrado {
4
5     private double lado;
6
7     public quadrado(double lado) {
8         this.lado = lado;
9     }
10    public double area(){
11        return lado*lado;
12    }
13
14    @Override
15    public String toString() {
16        return String.format("Quadrado \n Lado:%.2f \n Área:%.2f\n", lado, area());
17    }
18 }
19
20
```

No exemplo anterior, podemos ver que todos os componentes da classe foram definidos como publicos, mas seu atributo **lado** foi definido como **private**. Com esse tipo de definição qualquer classe com um objeto do tipo quadrado terá acesso a todos os componentes, exceto ao atributo **lado**, que só poderá ser alterado no momento da instanciação do objeto. Essa restrição de acesso proíbe, inclusive, que outras classes tenham acesso ao valor do atributo, que no exemplo só pode ser visualizado através do método toString.

Métodos set e get

Para manter o acesso restrito e mesmo assim possibilitar, a alteração e recuperação dos valores num atributo privado, temos que criar métodos que façam uma “mediação”.

```
1 package figurasplanas;
2
3 public class quadrado {
4
5     private double lado;
6
7     public quadrado(double lado) {
8         this.lado = lado;
9     }
10
11    public double getLado() {
12        return lado;
13    }
14
15    public void setLado(double lado) {
16        this.lado = lado;
17    }
18
19    public double area(){
20        return lado*lado;
21    }
22
23    @Override
24    public String toString() {
25        return String.format("Quadrado \n Lado:%.2f \n Área:%.2f\n", lado, area());
26    }
27 }
28
```

Dessa forma uma classe, com um objeto do tipo **quadrado** terá acesso ao atributo **lado**, através do método **setLado** para alterar e do método **getLado** para recuperar o valor do atributo **lado**.

O problema da declaração dos métodos de acesso, da forma como foram feitos no exemplo anterior, é que voltamos ao problema original de possibilitarmos que qualquer classe tenha acesso irrestrito aos valores do atributo. A diferença agora, é que através do método **set** poderemos controlar os valores que são passados ao atributo, já que as outras classes só podem realizar o acesso através desses métodos.

```
1 package figurasplanas;
2
3 public class quadrado {
4
5     private double lado;
6
7     public quadrado(double lado) {
8         setLado(lado);
9     }
10
11     public double getLado() {
12         return lado;
13     }
14
15     public void setLado(double lado) {
16         if (lado <= 0) this.lado = 1;
17         else this.lado = lado;
18         //this.lado = (lado <= 0 ? 1 : lado);
19     }
20
21     public double area() {
22         return lado * lado;
23     }
24
25     @Override
26     public String toString() {
27         return String.format("Quadrado \n Lado: %.2f \n Área: %.2f \n", lado, area());
28     }
29 }
30
```

No exemplo, podemos ver que só serão aceitos valores acima de zero para o atributo **lado**, sendo atribuído o valor 1 caso tente ser atribuído um valor não positivo.

Exercícios:

1 – Refaça a classe **tempo** redefinindo os atributos para **private** e criando os métodos **get's** e **set's** para cada um dos atributos.

2 – Refaça a classe **data** redefinindo os atributos para **private** e criando os métodos **get's** e **set's** para cada um dos atributos. Faça um método **dias** que retorne quantos dias se passaram do início do ano até a data guardada nos atributos.

3 – Faça uma classe **Equipe** com os atributos **nome**, **vitorias**, **empates** e **saldoDeGols**. Faça um método **totalDePontos**, que retorne a pontuação da equipe (3 pontos por vitória e 1 por empate). Todos os atributos devem de tipo **private**. Não esqueça o **construtor** e o **toString**.

4 – Faça uma classe **Produto** com os atributos **modelo**, **anoFabricacao**, **valor** e **decontoAVista**. Faça dois métodos um chamado **PrecoAVista** (que retorne o valor do produto considerando o desconto) e um método **PrecoAPrazo** (que receba o número de parcelas e retorne o valor acrescido de 2% por parcela). Todos os atributos devem de tipo **private**, e o **totalVendas** deve ser inicialmente definido como 0(zero) no construtor e seu valor deverá ser alterado por dois métodos **Venda** e **Estorno**. Não esqueça o **construtor** e o **toString**.